

NAME- JITEN MOURYA

UID-23BCC70050

SUB- ADBMS

EXP-10

➤ **AIM:** To demonstrate the ACID properties of database transactions (especially Atomicity and Consistency) by performing multiple inserts into the FeePayments table, handling failures using ROLLBACK, and ensuring the database remains in a consistent state.

➤ **THEORY:**

- Transactions in DBMS: A transaction is a sequence of SQL operations treated as a single unit. Either all operations succeed (COMMIT) or none (ROLLBACK).
- ACID Properties:
- Atomicity: Ensures all operations in a transaction are completed, or none are.
- Consistency: Database moves from one valid state to another.
- Isolation: Transactions do not interfere with each other.
- Durability: Once committed, changes are permanent.
- Use Case of Transactions:
- Insert multiple fee payment records.
- If any insert fails (e.g., duplicate payment_id or invalid data), the entire transaction is rolled back.
- SQL Commands Used:
- START TRANSACTION / BEGIN: Begin a transaction
- COMMIT: Save changes permanently
- ROLLBACK: Undo changes due to failure

➤ **CODES:**

```
CREATE TABLE FeePayments (  
    payment_id INT PRIMARY KEY,  
    student_name VARCHAR(100) NOT NULL,  
    amount DECIMAL(10,2) CHECK (amount > 0),  
    payment_date DATE NOT NULL  
);  
BEGIN;  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (1,  
'Ashish', 5000.00, '2024-06-01');  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (2,  
'Smaran', 4500.00, '2024-06-02');  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (3,  
'Vaibhav', 5500.00, '2024-06-03');  
COMMIT;  
BEGIN;  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (4,  
'Kiran', 5200.00, '2024-06-04');  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (2,  
'Sneha', -4700.00, '2024-06-08'); -- duplicate  
payment_id and negative amount (to force failure)  
ROLLBACK;  
BEGIN;  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (5,  
'Aditya', 4600.00, '2024-06-05');  
INSERT INTO FeePayments (payment_id,  
student_name, amount, payment_date) VALUES (6,
```

NULL, 4700.00, '2024-06-06'); -- NULL student_name
(violates NOT NULL constraint)

ROLLBACK;

-- After above transactions, only valid committed data
should exist:

SELECT * FROM FeePayments;

ROLLBACK;

SELECT * FROM FeePayments;

➤ **OUTPUTS:**

The screenshot shows a web-based SQL IDE interface. The top bar indicates the user is logged in as Jiten Mourya. The main area displays a MySQL database named 'db_43xxpadsc'. The table 'FeePayments' is defined with the following structure:

payment_id	student_name	amount	payment_date
1	Ashish	5000.00	2024-06-01T00:00:00.000Z
2	Smaran	4500.00	2024-06-02T00:00:00.000Z
3	Vaibhav	5500.00	2024-06-03T00:00:00.000Z
4	Kiran	5200.00	2024-06-04T00:00:00.000Z
5	Aditya	4600.00	2024-06-05T00:00:00.000Z

The SQL code in the editor shows the following statements:

```
1 CREATE TABLE FeePayments (  
2   payment_id INT PRIMARY KEY,  
3   student_name VARCHAR(100) NOT NULL,  
4   amount DECIMAL(10,2) CHECK (amount > 0),  
5   payment_date DATE NOT NULL  
6 );  
7 BEGIN;  
8 INSERT INTO FeePayments (payment_id, student_name, amount, payment_date) VALUES (1, 'Ashish', 5000.00, '2024-06-01');  
9 INSERT INTO FeePayments (payment_id, student_name, amount, payment_date) VALUES (2, 'Smaran', 4500.00, '2024-06-02');  
10 INSERT INTO FeePayments (payment_id, student_name, amount, payment_date) VALUES (3, 'Vaibhav', 5500.00, '2024-06-03');  
11 COMMIT;  
12 BEGIN;  
13 INSERT INTO FeePayments (payment_id, student_name, amount, payment_date) VALUES (4, 'Kiran', 5200.00, '2024-06-04');  
14 INSERT INTO FeePayments (payment_id, student_name, amount, payment_date) VALUES (2, 'Sneha', -4700.00, '2024-06-08'); -- d  
15 ROLLBACK;  
16
```

➤ **LEARNING OUTCOMES:**

1. Learned how to use **transactions** in SQL with START TRANSACTION, COMMIT, and ROLLBACK.
2. Understood **Atomicity**, ensuring all operations in a transaction succeed or none are applied.
3. Observed **Consistency**, maintaining valid database state even when transactions fail.
4. Gained experience handling **transaction failures** caused by constraint violations or duplicates.

